

Trabajo práctico N°3



Integrantes:

- Medina, Gabriel 14086
- Vasquez, Nicolas 14097
- Albanez, Luciano 13185
- Fattiboni, Álvaro 13201

Ejercicio 1:

Una inferencia es el proceso lógico mediante el cual, a partir de una o varias premisas conocidas en la base de conocimiento (KB), se deriva una nueva fórmula (conclusión). En lógica proposicional: si de KB se puede derivar ϕ mediante reglas de inferencia (por ejemplo modus ponens, resolución), decimos que $KB \models \phi$ (ϕ es consecuencia lógica de KB).

Ejercicio 2:

Hay dos enfoques relacionados:

- Semántico: verificar que ϕ es consecuencia de KB significa comprobar que en todas las interpretaciones (modelos) que satisfacen KB también se satisface ϕ . Esto es la definición semántica: $KB \models \phi$.

- Sintáctico: usar un algoritmo de inferencia (por ejemplo resolución en CNF, encadenamiento hacia adelante/atrás para cláusulas Horn) que pruebe ϕ a partir de las reglas de KB. Si el algoritmo es sonido (no deriva falsos) y completo (puede derivar todo lo que semánticamente se sigue), entonces la prueba sintáctica confirma la inferencia.

Ejercicio 3:

Base de conocimiento

Las reglas y hechos dados son:

R1: $b \wedge c \rightarrow a$

R2: $d \wedge e \rightarrow b$

R3: $g \wedge e \rightarrow b$

R4: $e \rightarrow c$

R5: d (hecho)

R6: e (hecho)

R7: $a \wedge g \rightarrow f$

3.1) Encadenamiento hacia adelante (forward chaining) para probar $a = \text{True}$

Procedimiento (encadenamiento hacia adelante): partimos de los hechos conocidos y aplicamos reglas cuyo antecedente esté satisfecho, añadiendo las conclusiones hasta que aparezca a o no haya más reglas aplicables.

Aplicación concreta:

Inteligencia Artificial 2025 - Grupo 5

1. Hechos iniciales: d (R5), e (R6).
2. Con R4: de $e \rightarrow c$. Ahora sabemos c .
3. Con R2: de $d \wedge e \rightarrow b$. Dado que tenemos d y e , inferimos b .
4. Con R1: de $b \wedge c \rightarrow a$. Ahora que tenemos b y c , inferimos a .

Por tanto, con encadenamiento hacia adelante llegamos a: $a = \text{True}$.

Propiedad requerida: Para asegurar que esta inferencia sea posible por encadenamiento hacia adelante, el algoritmo debe ser sonido y completo respecto a la clase de fórmulas (aquí la KB es un conjunto de cláusulas Horn). En particular: forward chaining es completo para cláusulas Horn.

3.2) Encadenamiento hacia atrás (backward chaining) para probar $a = \text{True}$

Procedimiento (encadenamiento hacia atrás): comenzamos con la meta a y buscamos reglas que concluyan a . Si una regla tiene $b \wedge c \rightarrow a$, entonces el procedimiento intenta probar b y c recursivamente. Si alguna submeta es un hecho conocido, se acepta; si no, se busca derivarla.

Aplicación concreta:

1. Queremos a . Solo R1 tiene a en el consecuente: requerimos probar b y c .
2. Probar c : usar R4 ($e \rightarrow c$). Entonces probamos e . e está hecho (R6) $\rightarrow$ OK.
3. Probar b : usar R2 ($d \wedge e \rightarrow b$). Necesitamos d y e . d es hecho (R5), e es hecho (R6) $\rightarrow$ OK.
4. Ambas submetas se prueban, por tanto a se prueba.

Propiedad requerida: backward chaining es completo para cláusulas Horn si se emplea una estrategia de búsqueda que evite ciclos (p. ej. memoización) y explore las reglas necesarias. Es decir: completitud y prevención de loops.

3.3) Forma Normal Conjuntiva (CNF) y demostración por contradicción de $a = \text{True}$

Cada implicación $p \wedge q \rightarrow r$ se convierte en la cláusula $(\neg p \vee \neg q \vee r)$. Transformamos las reglas/hechos a CNF (cláusulas):

1. R1: $\neg b \vee \neg c \vee a$
2. R2: $\neg d \vee \neg e \vee b$
3. R3: $\neg g \vee \neg e \vee b$
4. R4: $\neg e \vee c$
5. R5: d
6. R6: e
7. R7: $\neg a \vee \neg g \vee f$

Demostración por contradicción (resolución): asumimos $\neg a$ y probamos contradicción.
Cláusulas iniciales:

- (1) $\neg b \vee \neg c \vee a$
- (2) $\neg d \vee \neg e \vee b$
- (3) $\neg g \vee \neg e \vee b$
- (4) $\neg e \vee c$
- (5) d
- (6) e
- (7) $\neg a \vee \neg g \vee f$
- (8) $\neg a$ (supuesto para contradicción)

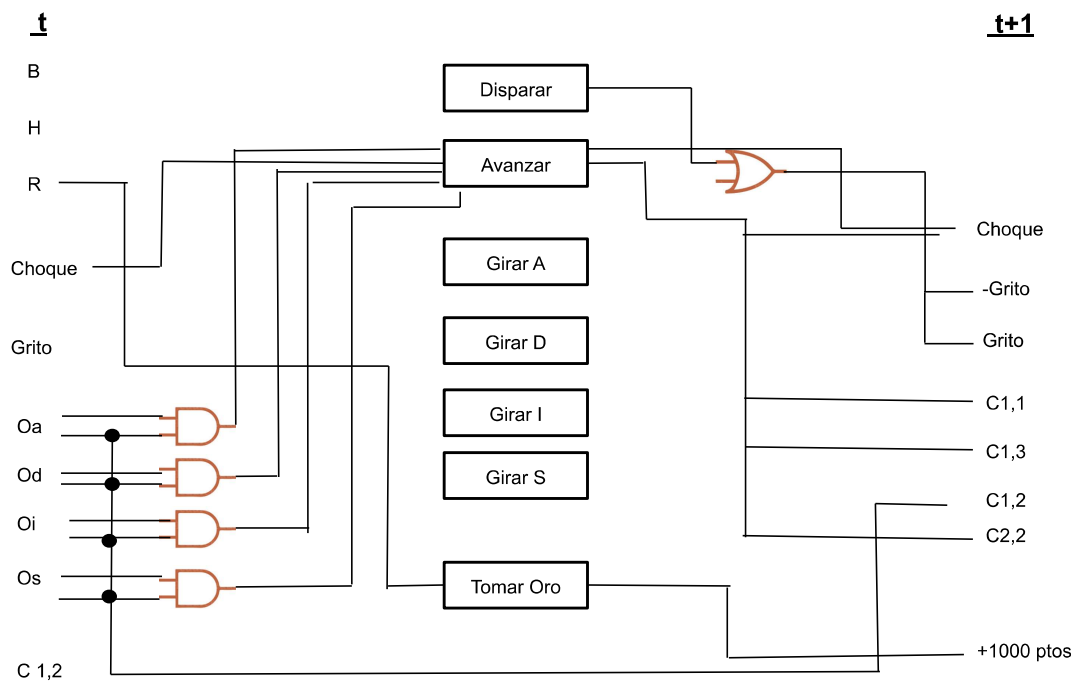
Resolución paso a paso:

- (2) con (5=d) $\Rightarrow$ resolvente (9): $\neg e \vee b$
- (9) con (6=e) $\Rightarrow$ resolvente (10): b
- (4) con (6=e) $\Rightarrow$ resolvente (11): c
- (1) con (10=b) $\Rightarrow$ resolvente (12): $\neg c \vee a$
- (12) con (11=c) $\Rightarrow$ resolvente (13): a
- (13) a con (8) $\neg a \Rightarrow$ cláusula vacía $\rightarrow$ contradicción.

Por tanto, la asunción $\neg a$ conduce a contradicción; concluimos que $a = \text{True}$.

Observación: no se necesitaron R3 ni R7 en esta demostración.

Ejercicio 4:



Ejercicio 5:

1. Pistas (enunciado)

Filas (de arriba hacia abajo):

F1: [1,1]

F2: [4]

F3: [2,1]

F4: [3]

Columnas (izquierda a derecha):

C1: [3]

C2: [3]

C3: [2,1]

C4: [3]

2. Reglas básicas del nonograma

Reglas usadas:

- Cada número indica la longitud de un bloque de casillas negras en esa fila/columna, en el orden dado.
- Los bloques sucesivos están separados por al menos una casilla blanca.
- Si la pista ocupa toda la fila/columna (ej. [4] en fila de 4), todas las casillas son blancas.
- Las casillas no marcadas explícitamente como negras se consideran potencialmente blancas hasta la demostración.

3. Reglas lógicas

- R1: $\text{Cas}(1,1) \wedge \text{Cas}(1,3) \vee \text{Cas}(1,2) \wedge \text{Cas}(1,4) \vee \text{Cas}(1,1) \wedge \text{Cas}(1,4)$
- R2: $\text{Cas}(2,1) \wedge \text{Cas}(2,2) \wedge \text{Cas}(2,3) \wedge \text{Cas}(2,4)$
- R3: $\text{Cas}(3,1) \wedge \text{Cas}(3,2) \wedge \text{Cas}(3,4) \vee \text{Cas}(3,1) \wedge \text{Cas}(3,3) \wedge \text{Cas}(3,4)$
- R4: $\text{Cas}(4,1) \wedge \text{Cas}(4,2) \wedge \text{Cas}(4,3) \vee \text{Cas}(4,2) \wedge \text{Cas}(4,3) \wedge \text{Cas}(4,4)$
- R5: $\text{Cas}(1,1) \wedge \text{Cas}(2,1) \wedge \text{Cas}(3,1) \vee \text{Cas}(2,1) \wedge \text{Cas}(3,1) \wedge \text{Cas}(4,1)$
- R6: $\text{Cas}(1,2) \wedge \text{Cas}(2,2) \wedge \text{Cas}(3,2) \vee \text{Cas}(2,2) \wedge \text{Cas}(3,2) \wedge \text{Cas}(4,2)$
- R7: $\text{Cas}(1,3) \wedge \text{Cas}(2,3) \wedge \text{Cas}(4,3) \vee \text{Cas}(1,3) \wedge \text{Cas}(3,3) \wedge \text{Cas}(4,3)$
- R8: $\text{Cas}(1,4) \wedge \text{Cas}(2,4) \wedge \text{Cas}(3,4) \vee \text{Cas}(2,4) \wedge \text{Cas}(3,4) \wedge \text{Cas}(4,4)$

Ahora podemos comenzar a inferir, y empezaremos por R2 ya que solo tiene una opción:

R2.1: $\text{Cas}(2,1)$

R2.2: $\text{Cas}(2,2)$

R2.3: $\text{Cas}(2,3)$

R2.4: $\text{Cas}(2,4)$

Inteligencia Artificial 2025 - Grupo 5

Fila 1: □ □ □ □

Fila 2: ■ ■ ■ ■

Fila 3: □ □ □ □

Fila 4: □ □ □ □

La notación será ■ para las casillas inferidas y □ para las que no, el tablero empezaría como un 4x4 negro.

Ahora nos movemos a R7 ya que de las dos opciones que hay solo una cumple con lo ya inferido para Cas (2,3):

R7.1: Cas (1,3)

R7.2: Cas (4,3)

Fila 1: □ □ ■ □

Fila 2: ■ ■ ■ ■

Fila 3: □ □ □ □

Fila 4: □ □ ■ □

Podemos ir ahora a R1 donde solo tenemos una posibilidad:

R1.1: Cas (1,1)

Fila 1: ■ □ ■ □

Fila 2: ■ ■ ■ ■

Fila 3: □ □ □ □

Fila 4: □ □ ■ □

Nos queda una sola posibilidad para R5:

R5.1: Cas (3,1)

Fila 1: ■ □ ■ □

Fila 2: ■ ■ ■ ■

Fila 3: ■ □ □ □

Fila 4: □ □ ■ □

Ya cumplimos con R7 por lo que Cas (3,3) no es posible, de esta manera nos queda una sola opción para R3:

R3.1: Cas (3,2)

R3.2: Cas (3,4)

Fila 1: ■ □ ■ □

Fila 2: ■ ■ ■ ■

Fila 3: ■ ■ □ ■

Inteligencia Artificial 2025 - Grupo 5

Fila 4: ☐ ☐ ☒ ☐

Por último, sabemos que como ya cumplimos con R5, Cas (4,1) no es posible nos queda una sola opción para R4

R4.1: Cas (4,2)

R4.2: Cas (4,4)

Fila 1: ☒ ☐ ☒ ☐

Fila 2: ☒ ☒ ☒ ☒

Fila 3: ☒ ☒ ☐ ☒

Fila 4: ☐ ☒ ☒ ☒

Construimos cumpliendo con R2, R7, R1, R5, R3 y R4. También podemos ver como gracias al cumplir con las mismas quedaron ya fijadas ciertas posibilidades para R6 y R8 las cuales son válidas, quedando como en lo anterior la tabla final.

Ejercicio 6:

```
1 # Clase que representa una regla de la base de conocimiento
2 class Regla:
3     def __init__(self, _condiciones, _conclusion):
4         self.condiciones = _condiciones # guarda la lista de condiciones (hechos necesarios)
5         self.conclusion = _conclusion    # guarda el hecho que se deduce si se cumplen las condiciones
6
7
8 # Clase que representa el motor de inferencia
9 class MotorInferencia:
10     def __init__(self, _reglas, _hechos_iniciales):
11         self.reglas = _reglas           # lista de objetos Regla
12         self.hechos = list(_hechos_iniciales) # copia los hechos iniciales en una lista interna
13
14     def encadenamiento_adelante(self):
15         nuevos = True                   # bandera para controlar si en una pasada se añadieron hechos nuevos
16         while nuevos:                   # repetir mientras se estén añadiendo hechos nuevos
17             nuevos = False              # asumimos que no habrá nuevos hechos hasta demostrar lo contrario
18
19             # mostrar los hechos actuales por pantalla (útil para depurar / entender el proceso)
20             print("\nHechos actuales:", self.hechos)
21
22             # recorreremos todas las reglas definidas en la base de conocimiento
23             for regla in self.reglas:
24                 # 'all(...)' comprueba que TODAS las condiciones de la regla estén en los hechos
25                 # No se necesita un bucle adicional aquí, se puede eliminar el for i,cond...
26                 V_bool = []
27                 for cond in regla.condiciones:
28                     V_bool.append(cond in self.hechos)
29
30                 if(all(V_bool)):
31                     # si la conclusión todavía no está entre los hechos, la añadimos
32                     if regla.conclusion not in self.hechos:
33                         self.hechos.append(regla.conclusion) # añade nuevo hecho
34                         print(f"Regla aplicada: {regla.condiciones} -> {regla.conclusion}")
35
36             nuevos = True # marcamos que sí hubo un cambio, para repetir el bucle
37
38     def mostrar_hechos(self):
39         print("\n=== Hechos finales ===")
40         for hecho in self.hechos:
41             print("-", hecho)
42
43 # -----
44 # Ejemplo de uso
45 # -----
46 if __name__ == "__main__":
47     # Definir la base de conocimiento (reglas)
48     r1 = Regla(["b", "c"], "a")
49     r2 = Regla(["d", "e"], "b")
50     r3 = Regla(["g", "e"], "b")
51     r4 = Regla(["e"], "c")
52     r5 = Regla(["a", "g"], "f")
53
54     reglas = [r1, r2, r3, r4]
55
```

En el código mostrado en la figura comenzamos definiendo una clase `Regla` que tiene como atributos a las condiciones de una sentencia y su resultado. Dicha clase se adopta como atributo de la clase `MotorInferencia` así como los hechos actuales, y aclaro actuales porque al iniciar uno le entrega los iniciales pero a medida que se van demostrando sentencias los hechos que tenemos cambian.

Como métodos de la `MotorInferencia` tenemos: `encadenamiento_adelante` y `mostrar_hechos`, la segunda tiene un comportamiento evidente, pinte todos los hechos encontrados, por lo que vamos a hacer hincapié en la segunda. El método `encadenamiento_adelante` comienza con una bandera que se encarga de verificar que no se hayan agregado nuevos hechos, ya que si no fue así significa que no hay más conclusiones que extraer con los hechos que tenemos. El `while` se encarga de evaluar el

flag, luego recorre todas las reglas de la lista vector compuesta por objetos de la clase **Regla** con un iterador llamado **regla**, luego definimos dentro del for un vector de booleanos que se llena dentro de otro for que se encarga de recorrer las condiciones de **regla** con otro iterador llamado **cond**, el cual toma como valor cada una de las condiciones de la regla y evalúa si esa condición se encuentra en los hechos actuales. Cuando todas las condiciones de la regla son hechos (evaluando esto último con `all(vector de booleanos)`), sabemos que la conclusión es verdadera y la agregamos a los hechos, proceso que se repite hasta demostrar todo lo que se puede con los hechos iniciales y reglas que tengamos. También agregamos un sumador que se encarga de analizar en cuántas iteraciones obtuvimos la respuesta definitiva.

Ejercicio 7:

1. Estructura general del programa

1. Definición de la base de reglas (reglas) — lista de tuplas (premisas, conclusión).
2. Inicialización: `contador_iteraciones = 0` y hechos (conclusiones de reglas sin premisas).
3. Función principal de inferencia: `probar_objetivo(objetivo, visitados=None, nivel=0)` — realiza la búsqueda recursiva hacia atrás y muestra la traza.
4. Menú interactivo: `menu()` — permite ver la base, solicitar la demostración de una hipótesis y salir.

Bloque `if __name__ == "__main__": menu()` — arranca el programa.

2. Representación de la BC

La base de conocimientos es una lista que tiene como elementos `__tuplas__`, estas tuplas tienen la forma (premisa, conclusion)

3. Inicialización de hechos

`hechos = set([conclusion for premisas, conclusion in reglas if len(premisas) == 0])`

- Esto crea un set con las conclusiones de reglas cuya lista de premisas está vacía.
- En tu base inicial `hechos = {'d', 'e'}` (R5 y R6).

Inteligencia Artificial 2025 - Grupo 5

- Nota: la comprensión desempaqueta cada tupla (premisas, conclusion) y por eso premisas aparece "sin haberse definido antes".

4. Funcionamiento de *probar_objetivo*

La función se comporta así (desglosado en pasos):

1. Incrementar contador (explicado arriba).
2. Inicializar visitados si es None: `visitados = set()`. Esto es el control de ciclo para la rama actual.
3. Impresión de traza: se usa nivel para producir una sangría `indent = " " * nivel` y luego `print(f"{indent}Intentando demostrar: {objetivo}")`.
4. Caso base rápido: si `objetivo in hechos` → imprime que es hecho conocido y devuelve `True`.
5. Detección de ciclos: si `objetivo in visitados` → ciclo detectado, imprime y devuelve `False`. Después se añade `visitados.add(objetivo)`.
6. Buscar reglas que concluyan el objetivo:
 - Se itera `for premisas, conclusion in reglas`: `if conclusion == objetivo`:
 - Si la regla tiene premisas imprime la forma `p1 ∧ p2 → objetivo`, si no (hecho base) lo indica.
7. Probar todas las premisas:
 - Se usa `all(probar_objetivo(p, visitados.copy(), nivel + 1) for p in premisas)`.
 - `all(...)` llamará a `probar_objetivo` para cada `p` hasta que encuentre una premisa que falle (corta evaluación).
 - Importante: se pasa `visitados.copy()` a cada premisa: cada rama de premisas recibe una copia del conjunto visitados, así cada sub-prueba no comparte nuevos marcados con las otras hermanas. Esto evita que una marca de una premisa afecte la evaluación de sus hermanas, pero también puede permitir que ciertas clases de ciclos entre premisas no se detecten. (Ver recomendaciones abajo.)
8. Si todas las premisas se demuestran:

- Se ejecuta `hechos.add(objetivo)` (memoización) y se imprime que se demostró, devuelve `True`.
9. Si ninguna regla pudo demostrar el objetivo: se imprime que no hay manera y devuelve `False`.

5. Conclusiones

El programa implementa correctamente un motor de encadenamiento hacia atrás con trazas por pantalla y un contador de llamadas. Su diseño es simple y efectivo para propósitos didácticos.

Ejercicio 8:

Introducción

El código implementa un **motor de inferencia basado en resolución**, un método de la lógica proposicional para demostrar proposiciones mediante **prueba por contradicción**.

La idea central es la siguiente:

1. Se convierte la base de conocimiento en **Forma Normal Conjuntiva (FNC)**.
2. Se agrega la **negación de la hipótesis** a demostrar.
3. Se aplican reglas de resolución entre cláusulas hasta:
 - Encontrar la **cláusula vacía** (contradicción), lo que implica que la hipótesis es verdadera.
 - O bien, no poder generar nuevas cláusulas, en cuyo caso la hipótesis no es demostrable con la base de conocimiento actual.

Este procedimiento es uno de los más conocidos en **sistemas de razonamiento automático** y en el campo de la **inteligencia artificial simbólica**.

Conceptos clave

Literal

Un **literal** es la unidad más básica en lógica proposicional. Puede ser:

- **Positivo**: una proposición como a , b , c .
- **Negativo**: la negación de una proposición, escrita como $\neg a$, $\neg b$, $\neg c$.

Ejemplo: En la proposición $\neg p \vee q$, los literales son $\neg p$ y q .

Cláusula

Una **cláusula** es una disyunción (OR) de literales.

En el programa, cada cláusula se representa como un conjunto de literales en FNC.

Ejemplo: $\{\neg b, \neg c, a\}$ corresponde a la proposición $\neg b \vee \neg c \vee a$.

Forma Normal Conjuntiva (FNC)

La **FNC** es una forma estandarizada en lógica proposicional donde toda fórmula se representa como una **conjunción (AND) de disyunciones (OR) de literales**.

Ejemplo: $(\neg b \vee \neg c \vee a) \wedge (\neg d \vee \neg e \vee b) \wedge (\neg e \vee c)$

Resolvente

El **resolvente** es una nueva cláusula que se obtiene aplicando la **regla de resolución** a dos cláusulas que contienen un literal complementario (un literal y su negación).

Ejemplo:

- Cláusula 1: $\{a, b\}$
- Cláusula 2: $\{\neg a, c\}$
- Resolvente: $\{b, c\}$ (porque a y $\neg a$ se eliminan).

Cláusula vacía

Si en el proceso de resolución se obtiene una cláusula vacía $\square$, significa que se encontró una **contradicción**. Esto prueba que la hipótesis es verdadera.

Estructura del código

El código define una clase principal `MotorInferenciaResolucion` que implementa todo el procedimiento. Sus componentes son:

1. **Base de conocimiento (BC)**: Definida en el constructor `__init__`, con un conjunto de reglas y hechos expresados en FNC. Ejemplo: $b \wedge c \rightarrow a$ se convierte en $\neg b \vee \neg c \vee a$.
2. **Método `es_literal_complementario`**: Comprueba si dos literales son opuestos (ejemplo: a y $\neg a$).
3. **Método `aplicar_resolucion`**: Genera un resolvente a partir de dos cláusulas con un único par de literales complementarios.
4. **Métodos de visualización (`mostrar_clausula`, `mostrar_base_conocimiento`)**: Muestran de manera legible las cláusulas.
5. **Método `demostrar_por_resolucion`**: Implementa paso a paso la prueba de una hipótesis:
 - Muestra la base de conocimiento.
 - Agrega la negación de la hipótesis.
 - Aplica resolución iterativamente.
 - Si encuentra la cláusula vacía, concluye que la hipótesis es verdadera.
 - Si no, concluye que no puede demostrarse.
6. **Método `demostrar_multiple`**: Permite probar varias hipótesis y resumir los resultados.
7. **Funciones `menu_interactivo` y `ejercicio_8`**: Ofrecen un menú de interacción con el motor o la ejecución automática de la demostración de la hipótesis a .

Ejemplo en el código

Inteligencia Artificial 2025 - Grupo 5

La base de conocimiento inicial incluye reglas como:

- $b \wedge c \rightarrow a$ (equivalente en FNC a $\neg b \vee \neg c \vee a$)
- $d \wedge e \rightarrow b$ (equivalente a $\neg d \vee \neg e \vee b$)
- $e \rightarrow c$ (equivalente a $\neg e \vee c$)
- Hechos: d, e

Con estas reglas, el motor puede demostrar si la hipótesis a es verdadera. El sistema añade $\neg a$ y aplica la resolución paso a paso hasta llegar a la contradicción, concluyendo que a es demostrable.

Conclusión

El programa implementa un **motor de inferencia por contradicción usando resolución**, que permite demostrar proposiciones de manera automática.

Es un ejemplo práctico de cómo funcionan los **sistemas expertos basados en lógica**, un pilar de la **inteligencia artificial simbólica**. Con él se pueden:

- Definir una base de conocimiento en FNC.
- Probar hipótesis.
- Extender la base con nuevas cláusulas.
- Observar el proceso de resolución paso a paso.

Este motor es una herramienta didáctica valiosa para comprender la **resolución en lógica proposicional** y su aplicación en sistemas de inferencia.

Conclusión Global: comparación entre los tres métodos.

Inteligencia Artificial 2025 - Grupo 5

Motor de inferencia	Iteraciones necesarias	Cláusulas creadas	Demostración de	Costo computacional (cualitativo)	Usos típicos o ideales
Encadenamiento hacia adelante	36	N/A	Todos los literales	Alto (muchas derivaciones innecesarias)	Ideal cuando se quiere derivar todo el conocimiento posible a partir de hechos iniciales (sistemas expertos, motores de reglas).
Encadenamiento hacia atrás	6	N/A	Solo a	Bajo/medio (se enfoca en una meta concreta)	Útil cuando se busca demostrar una hipótesis específica partiendo de la meta (diagnóstico médico, sistemas de consulta).
Por contradicción / resolución	3	45	Solo a	Alto (puede generar explosión combinatoria de cláusulas)	Muy potente en lógica formal, útil para demonstraciones automáticas de teoremas y en problemas donde es necesario un razonamiento exhaustivo.

En resumen:

- El **encadenamiento hacia adelante** es más costoso en iteraciones, pero produce un panorama completo.
- El **encadenamiento hacia atrás** es más eficiente para hipótesis puntuales.

Inteligencia Artificial 2025 - Grupo 5

- La **resolución** encuentra contradicciones rápidamente, pero puede generar un gran número de cláusulas intermedias.

Se incluye un PDF explicativo ("Implementación Motores Inferencia") sobre la implementación de los tres motores de inferencia. No es pseudocódigo, sino código "narrativo" que indica cómo sería el flujo de razonamiento en cada motor de inferencia.